

고객이 사용하는 AI 제품을 만들고 운영했습니다

고객 문제를 제품으로 정의하고, FastAPI 백엔드·AI 서비스·핵심 사용자 흐름을 직접 구현해 출시와 운영까지 연결한 경험을 정리했습니다.

각 사례는 먼저 누가 어떤 제품을 사용했는지 보여주고, 그 제품을 운영하기 위해 선택한 상태 관리와 실패 복구 방식을 뒤에서 설명합니다.

만들고 운영한 제품

제품의 사용 흐름과 운영 문제를 먼저 보고, 필요한 백엔드를 설계했습니다

AI 콘텐츠, 실시간 상담, 병원 운영·예약, 예약·결제처럼 사용자가 실제 업무를 끝내야 하는 제품을 만들었습니다. FastAPI는 제품을 완성하기 위해 선택한 주력 도구이며, 기술 사례는 각 제품에서 발생한 문제와 함께 설명합니다.

AI 콘텐츠 제품

사용자가 콘텐츠를 만들고 가져와 품질을 확인한 뒤 예약·발행하고 성과를 관리하는 유료 제품입니다.

[제품](#)[백엔드](#)[AI](#)

실시간 상담·주문/재고

상담 직원이 실시간 전사와 조언을 받고, 운영자는 주문·재고 작업의 실패를 확인하고 다시 처리하는 의료 서비스입니다.

[제품](#)[백엔드](#)[AI](#)

병원 운영·예약

여러 병원의 관리 업무와 홈페이지 예약 흐름을 지원하며, 예약을 개선을 통해 고객사 매출에 기여한 제품입니다.

제품

백엔드

운영

예약·결제

사용자가 예약과 선결제·취소·환불을 진행하고, 운영자가 알림과 처리 이력을 확인하는 고객 흐름을 구현했습니다.

제품

백엔드

운영

서로 다른 시기와 제품의 경험입니다. 아래 사례에서는 제품별 기여 범위와 구현 상태를 따로 표시했습니다.

제품과 백엔드 사례 · 5

01 콘텐츠 제작 문제를 실제 결제로 이어지는 AI 제품으로 풀었습니다

제품 운영을 리드하며 FastAPI 백엔드·독립 AI 서비스·Next.js 핵심 흐름을 직접 구현했고, 실제 고객이 결제하는 제품을 팀과 운영했습니다.

02 실시간 상담과 주문·재고 업무가 실패 뒤에도 이어지도록 만들었습니다

실시간 상담에서는 늦은 보정 결과의 대상을, 자동 발주에서는 알림 작업의 상태와 실패를 명시해 운영자가 추적하고 복구할 수 있게 했습니다.

03 여러 병원의 운영·예약을 지원하는 제품 백엔드를 구축하고 있습니다

관리·홈페이지 API를 분리한 백엔드 구조를 주도하고, 운영자의 작업 지점과 데이터 접근 범위는 서버가 결정하도록 보강하고 있습니다.

04 예약·선결제·환불·알림이 이어지는 고객 흐름을 구현했습니다

외부 결제와 내부 DB의 완료 시점을 분리하고, 예약 실패·환불·알림 변경이 남길 상태와 처리 순서를 구현했습니다.

05 여러 제품을 같은 기준으로 개발할 수 있는 FastAPI 기반을 만들었습니다

Python의 자유도를 프로젝트마다 다시 해석하지 않도록 계층·의존성·트랜잭션·비동기 세션 규칙을 템플릿과 문서에 고정했습니다.

콘텐츠 제작 문제를 실제 결제로 이어지는 AI 제품으로 풀었습니다

제품 운영을 리드하며 FastAPI 백엔드·독립 AI 서비스·Next.js 핵심 흐름을 직접 구현했고, 실제 고객이 결제하는 제품을 팀과 운영했습니다.

제품 사례

제품

백엔드

AI

운영

출시·운영

상황 · 사용자는 소셜 콘텐츠를 꾸준히 만들고 발행해야 했지만, 소재 수집·생성·품질 확인·예약·성과 확인이 여러 도구에 흩어져 있었습니다.

문제

생성 기능 하나만 제공해서는 실제 업무가 끝나지 않았습니다. 고객이 돈을 내는 이유를 찾으려면 콘텐츠 준비부터 발행 이후 관리까지 하나의 제품 흐름으로 이어야 했습니다.

대처 방법

- 기획·QA·마케팅과 고객 문제, 기능 우선순위, 생성 품질 기준을 정하고 출시·운영 흐름을 조율했습니다.
- FastAPI 제품 백엔드와 독립 AI 서비스·DB를 구축하고 인증된 HTTP로 연결했습니다.
- 콘텐츠 생성·가져오기·상태 확인·예약·발행·성과 확인·관리 화면 등 Next.js 핵심 흐름을 직접 구현했습니다.
- LLM 검수 결과는 1차 판단 이력으로 저장하고, 별도 관리자 라벨링 화면에서는 사람이 점수와 사유를 남기도록 자동 검수와 사람 평가의 역할을 분리했습니다.
- 제품 기준 데이터 변경과 Outbox를 같은 트랜잭션에 기록하고 AI 전달은 재시도와 버전 비교가 가능한 워커로 분리했습니다.

결과

실제 사용자가 이용하고 구독료 매출이 발생하는 AI 콘텐츠 제품으로 운영했습니다. 제품 운영은 기획·QA·마케팅과 함께했고, 제가 직접 맡은 구현 범위는 백엔드·AI 서비스·핵심 화면입니다.

사용자 업무와 제품 실행 흐름

구조

사용자 업무 흐름과 제품 백엔드·AI 실행 흐름을 나란히 두어, 기능이 실제 제품 경험으로 이어지는 경계를 표시했습니다.



기여 내용

해결 과정에서 맡은 기술 판단과 구현 범위입니다.

제품 판단

제품 범위

생성 결과가 아니라 예약·발행과 성과 확인까지 사용자가 마칠 수 있는 흐름을 제품 범위로 잡았습니다.

품질 기준

자동 검수는 점수·통과 여부·사유·개선안을 이력으로 남기고, 맥락과 자연스러움은 사람이 글마다 점수와 판단

직접 구현

제품 백엔드

콘텐츠·예약·발행·계정·관리 API와 제품 기준 데이터를 FastAPI로 구축했습니다.

AI 서비스

사유를 남기도록 역할을 나눴습니다.

생성 과정과 실행 상태를 별도 FastAPI 서비스·DB로 분리했습니다.

핵심 화면

사용자와 운영자가 실제로 쓰는 Next.js 흐름을 백엔드 계약과 함께 구현했습니다.

실패 복구

전달 실패

기존 데이터 변경과 Outbox를 함께 기록하고 워커 중단·중복·지연·역순 전달을 재시도와 버전 비교로 통제했습니다.

확인 가능한 실패

최대 재시도 뒤에도 실패한 항목을 남겨 운영자가 원인과 상태를 확인할 수 있게 했습니다.

기여 범위

제품 운영은 기획·QA·마케팅과 함께 수행했으며 제품 전체를 혼자 만들었다고 표현하지 않습니다.

구독료 매출은 제품 전체의 성과이며 정확한 금액은 공개 범위에서 제외했습니다.

담당 범위

리드 제품 운영과 직접 구현

기획·QA·마케팅과 제품 운영을 리드하고 FastAPI 백엔드·AI 서비스·Next.js 핵심 흐름을 직접 구현했습니다.

함께 수행 유료 제품 운영

실제 고객이 결제하고 구독료 매출이 발생하는 제품 운영에 함께 기여했습니다.

피노키오랩 지원 직무와의 연결

AI 기능을 사용자 업무가 끝나는 제품으로 연결

FastAPI 백엔드와 독립 AI 서비스의 출시·운영 경험

제품 문제와 기술 구현을 함께 다루는 역할

실시간 상담과 주문·재고 업무가 실패 뒤에도 이어지도록 만들었습니다

실시간 상담에서는 늦은 보정 결과의 대상을, 자동 발주에서는 알림 작업의 상태와 실패를 명시해 운영자가 추적하고 복구할 수 있게 했습니다.

반복 적용한 방식

제품

백엔드

AI

운영

별도 서비스 운영

상황 · 실시간 상담과 주문·재고는 같은 의료 플랫폼에 속하지만, 전자는 발화와 보정 결과의 대응을, 후자는 주문 생성 뒤 알림 작업의 상태를 다뤄야 하는 별도 서비스였습니다.

실시간 상담

문제

COMPLETE 뒤 보정 결과가 비동기로 도착하고 같은 문장이 반복될 수 있어, 텍스트나 도착 순서만으로는 어떤 발화를 바꿔야 하는지 보장할 수 없었습니다.

대처 방법

- DELTA·COMPLETE·CORRECTED를 같은 sequence로 연결
- sequence Map에서 해당 발화만 교체하고 세션 시작 시 Map 초기화
- 종료 시 타이머·세션·예약 작업을 함께 정리

결과

늦게 도착한 보정도 정확한 발화에 반영하고, 종료된 상담이 다시 연결되는 경로를 차단했습니다.

주문·재고

문제

자동 발주로 주문이 생성된 뒤 병원·공급사 알림은 별도로 실패할 수 있어, 발송 결과를 주문 처리 상태와 함께 관리해야 했습니다.

대처 방법

- 병원 묶음 알림과 공급사별 알림을 TaskIQ worker로 분리
- 공급사 알림 결과를 주문의 PENDING·FAILED 상태로 수렴
- 자동 재시도 뒤에도 실패한 주문의 수동 재발송 API 구현

결과

운영자가 알림 진행 상태와 실패 주문을 확인하고 조건을 검증한 뒤 다시 발송할 수 있게 했습니다.

실시간 이벤트의 순서 제어와 비동기 작업의 실패 복구를 한 화면에서 비교하되 서로 다른 서비스임을 분리했습니다.



기여 내용

해결 과정에서 맡은 기술 판단과 구현 범위입니다.

기술 선택 비동기 작업 외부 API·DB I/O 비중이 높고 여러 작업을 동시에 처리해야 해, FastAPI와 같은 asyncio 실행 모델을 유지하기로 했습니다. Celery의 제약 당시 사용하던 Celery에서는 async def 작업을 워커가 그대로 실행하는 공식 경로가 없어 별도의 이벤트 루프 관리가 필요했습니다. TaskIQ 선택 TaskIQ는 coroutine 작업과 RabbitMQ 연동을 공식 지원해 애플리케이션과 워커 사이의 동기·비동기 변환을 줄일 수 있다고 판단했습니다.	기여 범위 실시간 상담은 세션·전사 이벤트 흐름을 공동으로 맡았고, 주문·재고 백엔드는 구축을 주도했습니다. TaskIQ 전환 전후의 성능 지표는 측정하지 않아 구조 선택과 구현 범위만 설명합니다.
--	---

담당 범위

함께 수행 세션·전사 이벤트 흐름

세션 생명주기와 VAD·DELTA·COMPLETE·CORRECTED 흐름, 외부 STT 연동 안정화를 함께 담당했습니다.

리드 API·워커·복구 흐름

자동 발주 뒤 병원·공급사 알림을 RabbitMQ·TaskIQ 워커로 분리하고, 발송 결과를 주문 상태·재시도·수동 재발송과 연결했습니다.

피노키오랩 지원 직무와의 연결
실시간 AI 기능과 업무 상태를 함께 다룬 경험
비동기 작업의 재시도·최종 실패·수동 복구 설계
FastAPI 실행 모델에 맞춘 워커 기술 선택

여러 병원의 운영·예약을 지원하는 제품 백엔드를 구축하고 있습니다

관리·홈페이지 API를 분리한 백엔드 구조를 주도하고, 운영자의 작업 지점과 데이터 접근 범위는 서버가 결정하도록 보강하고 있습니다.

제품 사례

제품

백엔드

운영

제품 운영·백엔드 고도화 중

상황 · 여러 병원이 홈페이지 예약과 내부 관리 업무를 같은 제품에서 처리하며, 운영자는 소속 지점과 현재 작업 지점에 따라 다른 데이터를 봐야 합니다.

문제

관리 기능과 공개 홈페이지 기능의 변경 주기가 달랐고, 클라이언트가 보낸 지점 값만 믿으면 권한 밖 데이터가 조회될 수 있었습니다.

대처 방법

- 관리 API와 홈페이지 API를 독립 모듈로 두고 게이트웨이를 통해 하나의 진입점으로 제공했습니다.
- Clean Architecture 계층과 공통 저장소 패턴으로 업무 규칙과 데이터 접근을 분리했습니다.
- 운영자의 소속 지점과 현재 작업 지점을 나누고, 권한 검증을 통과한 서버 인증 상태에서 조회 범위를 결정하도록 구현하고 있습니다.
- 작업 지점 미선택과 권한 밖 접근을 409·403으로 구분해 클라이언트가 복구 방식을 선택하게 했습니다.

현재 진행

여러 병원의 운영·예약을 지원하는 제품 백엔드를 구축했고, 제품은 예약률 개선을 통해 고객사 매출 성과에 기여했습니다. 접근 범위의 전체 회귀 검증은 계속 보강 중입니다.

홈페이지 사용자와 운영자의 진입점을 분리하고, 운영자 데이터는 서버가 확인한 작업 범위로만 조회하도록 표시했습니다.



기존 홈페이지의 지점 식별 계약은 유지하면서 관리 API의 접근 경계만 변경했습니다.

작업 지점 접근의 전체 회귀 시나리오는 아직 보강 중입니다.

담당 범위

리드 관리·홈페이지 백엔드

관리·홈페이지 API를 독립 모듈로 둔 백엔드 구조와 구축을 주도하고 있습니다.

함께 수행 제품 성과와 접근 범위

제품은 예약률 개선을 통해 고객사 매출에 기여했고, 현재 작업 지점의 서버 기준 접근 경계를 보강하고 있습니다.

피노키오랩 지원 직무와의 연결

여러 사용자 역할과 데이터 범위를 다루는 제품 백엔드

공개 기능과 운영 기능의 모듈 경계

진행 중인 제품을 운영하며 구조를 고도화한 경험

예약·선결제·환불·알림이 이어지는 고객 흐름을 구현했습니다

외부 결제와 내부 DB의 완료 시점을 분리하고, 예약 실패·환불·알림 변경이 남길 상태와 처리 순서를 구현했습니다.

[제품 사례](#)[제품](#)[백엔드](#)[운영](#)[운영 경험](#)

상황 • 병원 예약 과정에서 사용자는 선결제하고, 예약 변경이나 실패가 생기면 환불과 내부 마일리지·이용권, 고객 알림이 함께 바뀌어야 했습니다.

문제

외부 결제사와 내부 DB를 하나의 트랜잭션으로 묶을 수 없고, 환불 요청과 실제 완료 시점도 달랐습니다. 알림 예약을 바꾸면 기존 작업과 새 작업의 이력도 함께 관리해야 했습니다.

대처 방법

- Stripe Checkout 선결제를 구축하고 내부 거래 ID를 결제 이력과 외부 이벤트에 연결했습니다.
- 예약 실패 시 결제 상태에 따라 취소·환불하고, 환불 완료 뒤 내부 마일리지·이용권을 변경하도록 순서를 보완했습니다.
- 다국어 알림톡·이메일의 즉시/예약 발송과 Celery 작업 취소·재등록·발송 이력을 구현했습니다.

결과

예약과 결제, 내부 자산, 고객 알림이 서로 다른 시점에 완료돼도 어떤 요청이 어디까지 처리됐는지 이력으로 확인할 수 있게 했습니다.

사용자의 예약 흐름과 외부 결제·내부 상태·알림 작업이 완료되는 순서를 나눠 표시했습니다.



예약 변경 시 기존 Celery 작업을 취소하고 새 작업과 발송 이력을 연결했습니다.

결제 시스템 전체를 구축하거나 운영을 전담한 사례가 아닙니다.

담당 범위

리드 예약·선결제

FastAPI 예약 API와 Stripe Checkout 선결제를 구현하고 내부 거래 ID로 결제 이력과 외부 이벤트를 연결했습니다.

함께 수행 환불·알림 상태

환불 완료 뒤 내부 상태가 바뀌도록 처리 순서를 보완하고, 알림 즉시/예약 발송과 작업 이력을 구현했습니다.

피노키오랩 지원 직무와의 연결

외부 서비스와 내부 업무 상태를 연결한 경험

요청·완료·보상 시점이 다른 흐름의 정합성 관리

예약·결제·알림이 이어지는 고객 기능 구현

여러 제품을 같은 기준으로 개발할 수 있는 FastAPI 기반을 만들었습니다

Python의 자유도를 프로젝트마다 다시 해석하지 않도록 계층·의존성·트랜잭션·비동기 세션 규칙을 템플릿과 문서에 고정했습니다.

반복 적용한 방식

백엔드

운영

조직 표준 직접 구축

상황 · 소수의 백엔드 인원이 여러 제품을 맡는 가운데, 기획·QA·디자인 담당자는 코딩 에이전트로 사내 프로그램을 만들고 있었습니다.

문제

Router의 session을 모든 계층에 전달하며 업무와 무관한 인자가 반복됐고, transaction을 끊거나 중첩할 위치도 호출부마다 정해야 했습니다. 하위 태스크의 AsyncSession 공유는 동시 접근과 롤백 범위도 흐렸습니다.

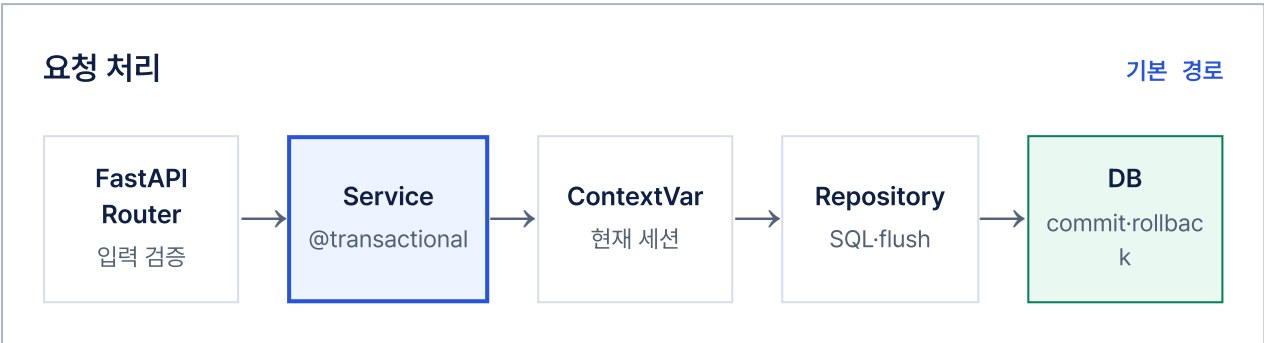
대처 방법

- Layered Architecture·Service Layer·Repository Pattern과 DI로 계층별 책임을 고정했습니다.
- @transactional이 메서드의 선언에 따라 REQUIRED·REQUIRES_NEW·NESTED를 처리하게 했습니다.
- ContextVar·SessionProxy로 현재 AsyncSession을 찾고, 트랜잭션당 asyncio 태스크 하나만 허용했습니다.
- 롤백·연결 정리·전파·세션 생명주기를 통합 테스트로 검증하고 ADR에 선택 기준을 남겼습니다.

결과

기획·QA·디자인 담당자가 코딩 에이전트로 사내 프로그램을 만들 때도 같은 계층·트랜잭션·검증 기준을 쓰도록 FastAPI 템플릿과 작업 지침을 제공했습니다.

요청이 서비스 계층의 트랜잭션 경계를 지나 현재 세션을 찾는 흐름과, 중첩 호출에서 선택할 세 가지 전파 방식을 분리했습니다.



FastAPI의 비동기 처리 능력은 유지하되, 하나의 DB 트랜잭션 안에서는 한 태스크와 한 AsyncSession만 사용하도록 범위를 좁혔습니다.

실행 방식

중첩 호출에서 선택하는 트랜잭션 동작

서비스 메서드는 필요한 동작만 선언하고 데코레이터가 참여·분리·SAVEPOINT를 처리합니다.

구분	설계한 동작	실패 경계
REQUIRED	기존 트랜잭션에 참여하고, 없으면 새 최상위 트랜잭션을 시작	같은 세션 · 바깥 작업과 함께 롤백
REQUIRES_NEW	새 연결·AsyncSession에서 독립 실행하고 종료 뒤 바깥 작업을 재개	독립 commit/rollback · 연결 비용 확인
NESTED	같은 연결에서 SAVEPOINT를 생성	내부 작업 롤백 · 바깥 작업은 계속

기여 내용

해결 과정에서 많은 기술 판단과 구현 범위입니다.

구조 책임 분리 Router → Service → Validator → Repository → Model의 책임을 정의하고 DI로 조립했습니다.	트랜잭션 REQUIRED 기존 트랜잭션에 참여하고 없으면 새 최상위 트랜잭션을 시작합니다.
선택 이유 Java/Spring에서 익숙한 명시적 책임과 호출 규칙을 적용해 팀원이 케이스마다 구조를 다시 정하는 비용을 줄이고자 했습니다.	REQUIRES_NEW 기존 트랜잭션을 잠시 두고 새 연결과 AsyncSession에서 독립 실행합니다.
비동기 세션 현재 세션 ContextVar에 AsyncSession과 트랜잭션 상태를 연결하고 SessionProxy가 현재 세션을 찾도록 했습니다.	선택의 대가 하나의 트랜잭션 안에서 같은 AsyncSession을 공유하는 내부 병렬 DB 작업은 허용하지 않습니다.
반복 제거 Router에서 받은 session을 Service-Validator-Repository의 모든 메서드에 계속 전달하지 않아도 되게 했습니다.	병렬 작업이 필요하면 태스크별 트랜잭션과 데이터 가시성 ·복구·연결 비용을 먼저 정해야 합니다.
하위 태스크 차단 같은 AsyncSession을 하위 asyncio 태스크가 사용하면 즉시 실패시켜 트랜잭션 경계를 분명히 했습니다.	

담당 범위

FastAPI·SQLAlchemy 실행 기준

계층·DI·트랜잭션·ContextVar·SessionProxy·소유 태스크 확인과 통합 테스트를 조직 표준 템플릿으로 직접 구축했습니다.

팀 적용 기준

기획·QA·디자인 담당자가 코딩 에이전트로 사내 프로그램을 구현할 때도 같은 기본값과 검증 명령을 읽도록 ADR·운영 절차서·작업 지침을 함께 제공했습니다.

피노키오랩 지원 직무와의 연결

- FastAPI·SQLAlchemy 비동기 실행 모델의 실제 설계 경험
- 여러 개발자가 이해하고 유지할 수 있는 계층·트랜잭션 기준
- 코딩 에이전트 활용 환경에서도 검증 가능한 기본 구조

제품을 정확하게 만드는 방법을 매주 함께 점검했습니다

1인 1제품 개발 환경에서 주 1회 에이전트 활용 경험과 작업 중 막힌 지점을 공유했습니다.
코드 자체보다 제품이 풀어야 할 문제, 구현 방향, 새 도구와 방법을 어디에 적용할지 검토했습니다.

공유 기준 모두가 읽는 맥락	만들어야 할 것	→ 문제·요구사항 확인	→ 구현 방향 비교	→ 적용 방법 결정
탐색·구현 에이전트 활용 코드베이스 조사·반복 작업 ·대안 정리	현재 작업 맥락	→ 막힌 지점과 원인	→ 새로운 방법 후보	
판단·승인 팀 검토 문제 정의·구현 방향·출시 여부	만들어야 할 것 확정	→ 구현 방향 비교	→ 적용·배포 결정	

생성 품질은 자동 검수와 사람 라벨링을 나눠 운영했습니다

자동 검수는 빠른 1차 판단과 이력을 맡고, 사람이 글의 맥락과 자연스러움을 평가해 다음 기준을 보완했습니다.

시스템 확인 01	실측 확인 02	사람이 결정 03
자동 검사	실제 결과 대조	최종 판단
틀리지 않았는가	기준에 부합하는가	실제로 통하는가
형식·계약 위반 확인 LLM 점수·통과 여부·사유· 개선안 저장	Threads corpus를 독립 labeling schema로 적재 평가 진행률과 미평가 대상을 조회	관리자 화면에서 글별 1~10점 기록 판단 사유를 남기고 다음 글로 이동

최종 판단 LLM 검수와 사람 라벨은 서로 다른 경계입니다. 자동 판정이 사람의 품질 라벨을 대신한다고 표현하지 않습니다.

■ 실제 적용 범위	주간 제품 개발 회고 문제 정의·구현 방향 검토	코드 단위 리뷰보다 무엇을 만들어야 하는지와 구현 방향· 효율적인 방법·작업 병목을 팀과 논의했습니다.
	백엔드 템플릿 사람과 에이전트가 읽는 개발 기준	아키텍처·운영 절차서·검증 명령·에이전트 작업 지침을 함께 구축했습니다.

에이전트가 만든 코드의 양보다 제품 요구를 정확히 이해하고 검증 가능한 방법으로 구현했는지를 팀이 함께 확인한 회고입니다.